

SQL, SQLITE И SQLALCHEMY В PYTHON

ИНТЕРАКТИВНЫЙ ГАЙД.....	2
Модуль 1: Базовый SQL и прямая работа с SQLite.....	2
1.1 Чек-лист знаний.....	2
1.2 Глоссарий.....	2
1.3 Полезные ссылки.....	3
1.4 Задача на проверку.....	3
Модуль 2: Введение в SQLAlchemy (Core & ORM).....	4
2.1 Чек-лист знаний.....	4
2.2 Глоссарий.....	4
2.3 Полезные ссылки.....	4
2.4 Задача на проверку.....	4
Модуль 3: Связи между таблицами и сложные запросы.....	6
3.1 Чек-лист знаний.....	6
3.2 Справочная таблица методов и операторов.....	6
3.3 Полезные ссылки.....	8
3.4 Задача на проверку.....	8
СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ.....	9
Тест 1: Прямая работа с SQLite через sqlite3.....	9
Часть А: Выберите один правильный ответ.....	9
Часть Б: Задания со свободным ответом (команды / код).....	9
Тест 2: Инициализация SQLAlchemy и описание моделей.....	11
Часть А: Выберите один правильный ответ.....	11
Часть Б: Задания со свободным ответом (команды / код).....	11
Тест 3: CRUD-операции и связи в SQLAlchemy.....	13
Часть А: Выберите один правильный ответ.....	13
Часть Б: Задания со свободным ответом (команды / код).....	13
ОТВЕТЫ И РАЗБОР ЗАДАНИЙ.....	15
Тест 1: Прямая работа с SQLite через sqlite3.....	15
Тест 2: Инициализация SQLAlchemy и описание моделей.....	16
Тест 3: CRUD-операции и связи в SQLAlchemy.....	17

ИНТЕРАКТИВНЫЙ ГАЙД

Здесь всё, что нужно знать, помнить и уметь применять на автомате при работе с базами данных в Python. Если вы запутались в сессиях, транзакциях или связях – это ваш чек-лист для самопроверки и ликвидации пробелов.

Модуль 1: Базовый SQL и прямая работа с SQLite

Что нужно уметь: Создавать подключение к базе данных SQLite с помощью встроенного модуля `sqlite3`, писать сырые SQL-запросы для создания таблиц, вставки, чтения и обновления данных, а также корректно закрывать соединения и фиксировать транзакции.

1.1 Чек-лист знаний

- [] Понимание разницы между реляционными БД (хранение в виде таблиц) и нереляционными.
- [] Подключение к файлу базы данных или создание БД в оперативной памяти (`:memory:`).
- [] Роль объекта `cursor` (курсор) как инструмента для выполнения команд и извлечения результатов.
- [] Написание базовых SQL-запросов: `CREATE TABLE`, `INSERT INTO`, `SELECT`, `UPDATE`, `DELETE`.
- [] Разница между выполнением одного запроса (`execute()`) и пакетной вставкой (`executemany()`).
- [] Управление транзакциями: фиксация изменений (`commit()`) и откат в случае ошибки (`rollback()`).
- [] Безопасная передача параметров в SQL-запрос через плейсхолдеры (`?`) для защиты от SQL-инъекций.

1.2 Глоссарий

- **База данных (Database)** – структурированный массив данных, хранящийся в электронном виде. В случае SQLite вся база данных – это один файл на диске.
- **Курсор (Cursor)** – специальный объект управления, который позволяет отправлять запросы к БД и поочередно извлекать строки с результатами.
- **SQL-инъекция (SQL Injection)** – уязвимость, при которой злоумышленник внедряет вредоносный SQL-код в запрос через

пользовательский ввод. Лечится только параметризацией запросов.

- **Транзакция (Transaction)** – логическая единица работы с БД, состоящая из одного или нескольких запросов, которая либо выполняется целиком (`commit`), либо полностью отменяется (`rollback`).

1.3 Полезные ссылки

- Официальная документация: [Модуль sqlite3 в Python](#)
- Интерактивный тренажер SQL: [SQLBolt \(на английском\)](#)

1.4 Задача на проверку

"Электронная библиотека"

Напишите скрипт на Python, который с помощью модуля `sqlite3` создает базу данных `library.db`. В ней должна быть таблица `books` с полями `id` (автоинкремент), `title` (текст), `author` (текст) и `year` (целое число).

1. Добавьте в таблицу 3 любые книги, используя параметризованный запрос.
2. Выберите из базы и выведите на экран все книги, выпущенные после 2010 года.
3. Не забудьте зафиксировать транзакцию и закрыть соединение.

Модуль 2: Введение в SQLAlchemy (Core & ORM)

Что нужно уметь: Инициализировать движок (Engine), настраивать фабрику сессий, объявлять структуру таблиц с помощью современной декларативной системы (Declarative Base) в SQLAlchemy 2.0 и выполнять простейшие CRUD-операции через ORM.

2.1 Чек-лист знаний

- [] Понимание концепции ORM (Object-Relational Mapping) как моста между объектами Python и строками в БД.
- [] Создание точки подключения к БД через `create_engine()` и настройка логирования (`echo=True`).
- [] Создание базового класса для моделей с помощью `DeclarativeBase` (современный стиль SQLAlchemy 2.0).
- [] Объявление полей таблицы с использованием `Mapped` и `mapped_column()` (определение типов, `primary_key`, `nullable`).
- [] Управление жизненным циклом сессии с помощью контекстного менеджера `sessionmaker` и объекта `Session`.
- [] Добавление объектов в базу (`session.add()`), сохранение изменений (`session.commit()`) и получение записей по первичному ключу (`session.get()`).

2.2 Глоссарий

- **Engine (Движок)** – центральный компонент SQLAlchemy, который хранит пул соединений (Connection Pool) и знает, как транслировать команды Python в диалект конкретной БД.
- **Сессия (Session)** – объект, который управляет транзакциями и отслеживает изменения во всех объектах ORM, загруженных или созданных в рамках текущей рабочей единицы (Unit of Work).
- **Mapped / mapped_column** – инструменты аннотации типов и конфигурации колонок в SQLAlchemy 2.0, связывающие атрибуты класса Python с типами данных в SQL.

2.3 Полезные ссылки

- Официальный учебник: [SQLAlchemy 2.0 Unified Tutorial](#)
- Архитектура: [Разница между Core и ORM в SQLAlchemy](#)

2.4 Задача на проверку

"Учет сотрудников"

Используя SQLAlchemy 2.0, опишите модель класса `Employee`, которая соответствует таблице `employees`. Поля: `id` (int, первичный ключ), `name` (str, обязательное), `salary` (int, по умолчанию 50000).

Напишите код, который:

1. Создает движок для SQLite в памяти (`sqlite3:///memory:`).
2. Создает таблицу в базе данных.
3. Открывает сессию, добавляет нового сотрудника по имени "Иван" со стандартной зарплатой и фиксирует транзакцию.
4. В этой же сессии находит этого сотрудника по ID и выводит его имя на экран.

Модуль 3: Связи между таблицами и сложные запросы

Что нужно уметь: Проектировать связи «один ко многим» (One-to-Many) с использованием внешних ключей (`ForeignKey`) и `relationship()`, писать запросы со сложной фильтрацией, сортировкой и объединением таблиц (`JOIN`).

3.1 Чек-лист знаний

- [] Настройка связи «один ко многим» с помощью `ForeignKey("parent_table.id")` на стороне дочерней таблицы.
- [] Использование функции `relationship()` для двунаправленной связи между объектами (параметр `back_populates`).
- [] Написание запросов на чтение данных в стиле SQLAlchemy 2.0 с использованием функции `select()`.
- [] Применение фильтрации данных через методы `.where()`, `.filter_by()` и логические операторы.
- [] Объединение таблиц с помощью метода `.join()` для выборки связанных данных.
- [] Ограничение и пропуск записей (`.limit()`, `.offset()`) для реализации пагинации.
- [] Сортировка результатов через метод `.order_by()` (использование функций `desc()` и `asc()`).

3.2 Справочная таблица методов и операторов

Метод / Оператор	Что делает	Пример использования
<code>select(Model)</code>	Начинает построение SQL-запроса SELECT для указанной модели	<code>stmt = select(User)</code>

<code>session.scalars(stmt)</code>	Выполняет запрос и возвращает итерируемый список чистых объектов ORM	<code>users = session.scalars(stmt).all()</code>
<code>.where(Condition)</code>	Добавляет условие фильтрации (аналог <code>WHERE</code> в SQL)	<code>stmt.where(User.age > 18)</code>
<code>.join(Model.relation)</code>	Выполняет SQL <code>INNER JOIN</code> на основе настроенных связей моделей	<code>select(Post).join(Post.author)</code>
<code>.order_by(Column)</code>	Сортирует результат по указанной колонке	<code>stmt.order_by(User.name.desc())</code>
<code>ForeignKey("table.id")</code>	Устанавливает ограничение внешнего ключа на уровне базы данных	<code>mapped_column(ForeignKey("users.id"))</code>

<code>relationship(...)</code>	Создает виртуальное свойство в объекте Python для удобного доступа к связям	<code>posts: Mapped[list["Post"]] = relationship(back_populates="author")</code>
--------------------------------	---	--

3.3 Полезные ссылки

- Документация: [Работа со связями в SQLAlchemy](#)
- Руководство: [Построение запросов \(Select\) в 2.0](#)

3.4 Задача на проверку

"Блог и авторы"

Создайте две связанные модели: `User` (пользователь) и `Post` (статья). У одного пользователя может быть много статей (`User.posts`), а у каждой статьи есть ровно один автор (`Post.author`).

1. Добавьте внешнюю связь через `ForeignKey` и настройте `relationship(back_populates=...)` в обоих классах.
2. Напишите запрос, который находит все статьи автора с именем "admin" (используйте `.join()`).
3. Отсортируйте полученные статьи по дате создания или ID в обратном порядке.

СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ

Тест 1: Прямая работа с SQLite через sqlite3

Часть А: Выберите один правильный ответ

1. Какая функция модуля `sqlite3` используется для открытия или создания файла базы данных?
 - а) `sqlite3.open()`
 - б) `sqlite3.connect()`
 - в) `sqlite3.create()`
 - г) `sqlite3.start()`
2. Что делает метод `cursor.fetchall()`?
 - а) Удаляет все данные из таблицы.
 - б) Фиксирует текущую транзакцию в базе данных.
 - в) Возвращает все строки результата выполнения SQL-запроса в виде списка кортежей.
 - г) Закрывает соединение с базой.
3. Как правильно передать параметры в метод `execute()`, чтобы защититься от SQL-инъекций?
 - а) С помощью f-строк: `cursor.execute(f"SELECT * FROM users WHERE name='{name}')`
 - б) С помощью конкатенации строк: `cursor.execute("SELECT * FROM users WHERE name='" + name + "')`
 - в) Через кортеж параметров и знак `?`: `cursor.execute("SELECT * FROM users WHERE name = ?", (name,))`
 - г) Параметры нельзя передавать напрямую, нужно использовать только файлы конфигурации.
4. Зачем нужен вызов `connection.commit()`?
 - а) Для проверки синтаксических ошибок в коде.
 - б) Для сохранения всех изменений, сделанных в рамках текущей транзакции, на диск.
 - в) Для отмены всех последних операций.
 - г) Для очистки кэша курсора.
5. Какое значение аргумента нужно передать в функцию подключения, чтобы создать временную базу данных исключительно в оперативной памяти компьютера?
 - а) `":ram:"`
 - б) `":temp:"`
 - в) `":memory:"`
 - г) `None`

Часть Б: Задания со свободным ответом (команды / код)

6. Напишите команду создания объекта курсора из объекта соединения `conn`.
7. Исправьте синтаксическую ошибку в коде параметризованного запроса (передается один параметр): `cursor.execute("SELECT * FROM products WHERE id = ?", (product_id))`
8. Какую команду нужно выполнить у объекта соединения, если в процессе выполнения пачки запросов произошла ошибка и нужно полностью откатить транзакцию назад?
9. Напишите SQL-запрос для удаления таблицы с именем `old_logs`, если она существует.
10. Какой метод объекта курсора эффективнее использовать для вставки списка из 100 кортежей данных одной командой вместо прогона цикла?

Тест 2: Инициализация SQLAlchemy и описание моделей

Часть А: Выберите один правильный ответ

1. Что такое Engine (Движок) в экосистеме SQLAlchemy?
 - а) Графический интерфейс для просмотра таблиц.
 - б) Объект, управляющий соединениями с базой данных и кодированием SQL-запросов под конкретную СУБД.
 - в) Специальный тип данных для хранения больших чисел.
 - г) Модуль для автоматической генерации админ-панелей.
2. Какой класс в современном стиле SQLAlchemy 2.0 должен служить родителем для всех пользовательских ORM-моделей?
 - а) `BaseModel`
 - б) `DeclarativeBase`
 - в) `sessionmaker`
 - г) `MetaData`
3. Как указать, что колонка в таблице является первичным ключом (`PRIMARY KEY`) при описании модели в SQLAlchemy 2.0?
 - а) `id: int = primary_key()`
 - б) `id: Mapped[int] = mapped_column(primary_key=True)`
 - в) `id = Column(Integer, primary=True)`
 - г) `id: int = Field(primary_key=True)`
4. Что делает функция `Base.metadata.create_all(engine)`?
 - а) Удаляет все существующие таблицы в базе данных.
 - б) Проверяет типы данных в Python-коде на соответствие стандарту PEP 8.
 - в) Сканирует все объявленные модели-наследники Base и автоматически создает соответствующие им таблицы в БД, если их там еще нет.
 - г) Экспортирует структуру таблиц в JSON-файл.
5. Чем отличается использование `Session` в качестве контекстного менеджера (`with Session(engine) as session:`) от ручного открытия сессии?
 - а) Контекстный менеджер работает медленнее.
 - б) Контекстный менеджер автоматически гарантирует закрытие сессии (`close`) и очистку ресурсов по завершении блока кода.
 - в) Ничем не отличается, это просто декоративное форматирование.
 - г) В блоке контекстного менеджера нельзя делать запросы на чтение (`SELECT`).

Часть Б: Задания со свободным ответом (команды / код)

6. Напишите код импорта функции создания движка (Engine) из пакета `sqlalchemy`.
7. Объявите базовый класс `Base` для декларативных моделей в стиле SQLAlchemy 2.0.
8. Напишите объявление поля `email`, которое должно быть строкой, строго уникальной (`unique`) и обязательной для заполнения (`nullable=False`).
9. Какой метод объекта сессии используется для явного внесения изменений (транзакции) в базу данных?
10. Напишите строку подключения (Database URL) для создания локального файла базы данных SQLite с именем `app.db` в текущей папке проекта.

Тест 3: CRUD-операции и связи в SQLAlchemy

Часть А: Выберите один правильный ответ

1. Как выглядит правильный синтаксис базового запроса на выборку всех пользователей из модели `User` в стиле SQLAlchemy 2.0?
 - а) `session.query(User).all()`
 - б) `select(User).all()`
 - в) `stmt = select(User)`
 - г) `User.objects.all()`
2. Какой метод сессии позволяет максимально быстро и безопасно извлечь запись из базы данных по её первичному ключу (ID)?
 - а) `session.fetch_one(id)`
 - б) `session.get(Model, id)`
 - в) `session.select_by_id(id)`
 - г) `session.find(id)`
3. Какая функция используется для связи объектов на уровне Python-кода (чтобы можно было писать `user.posts`) и не создает физическую колонку в БД?
 - а) `ForeignKey()`
 - б) `relationship()`
 - в) `join()`
 - г) `connect()`
4. Что произойдет с объектом, если вызвать `session.add(user)`, но не вызвать `session.commit()` перед закрытием сессии?
 - а) Данные сохранятся в базе автоматически.
 - б) База данных заблокируется до перезагрузки компьютера.
 - в) Изменения будут полностью потеряны, так как транзакция не была зафиксирована.
 - г) Программа упадет со смертельной синтаксической ошибкой.
5. Зачем нужен метод `.scalars()` при обработке результатов запроса в SQLAlchemy 2.0?
 - а) Он переводит все числа из базы в тип `float`.
 - б) Он распаковывает внутреннюю структуру строк SQLAlchemy и возвращает чистые объекты ORM-моделей (вместо кортежей из одного элемента).
 - в) Он шифрует данные перед выводом в консоль.
 - г) Он удаляет дубликаты из выборки.

Часть Б: Задания со свободным ответом (команды / код)

6. Напишите запрос, который фильтрует пользователей из модели `User`, оставляя только тех, у кого уровень доступа `role == 'admin'`.

7. Как отсортировать результаты запроса `select(Product)` по цене (`Product.price`) в порядке убывания (от дорогих к дешевым)?
Напишите часть с методом сортировки.
8. Напишите синтаксис удаления объекта `order` из базы данных через сессию.
9. Какой параметр функции `relationship` отвечает за взаимное обновление связей на противоположной модели (обеспечивает двунаправленность)?
10. Напишите код, который с помощью метода `.limit()` и `.offset()` выбирает ровно 5 записей, пропуская первые 10.

ОТВЕТЫ И РАЗБОР ЗАДАНИЙ

Тест 1: Прямая работа с SQLite через sqlite3

1. б) `sqlite3.connect()` – это стандартная точка входа, возвращающая объект соединения. Если файла нет, он будет создан автоматически.
2. в) Метод возвращает список кортежей. Если строк нет – вернет пустой список.
3. в) Использование знака `?` заставляет драйвер безопасно экранировать любые входящие данные, полностью нивелируя риск SQL-инъекций.
4. б) Без фиксации (`commit`) все ваши `INSERT` или `UPDATE` останутся в памяти и сбросятся при закрытии сессии подключения.
5. в) `":memory:"` – стандартная служебная строка для разворачивания быстрой БД в ОЗУ. Идеально для тестов.
6. `cursor = conn.cursor()`
7. Параметр в кортеже должен оканчиваться запятой, иначе Python посчитает это просто математическими скобками вокруг числа: `(product_id,)`. Правильно: `cursor.execute("SELECT * FROM products WHERE id = ?", (product_id,))`
8. `conn.rollback()`
9. `DROP TABLE IF EXISTS old_logs;`
10. `cursor.executemany()` – этот метод принимает шаблон запроса и список кортежей, пакуя их в один эффективный пакет операций.

Тест 2: Инициализация SQLAlchemy и описание моделей

1. б) Движок – это рабочая лошадка, скрывающая под капотом особенности синтаксиса конкретной СУБД (PostgreSQL, MySQL, SQLite).
2. б) **DeclarativeBase** – современный подход, внедренный в SQLAlchemy 1.4/2.0 для полноценной поддержки аннотаций типов IDE.
3. б) Комбинация **Mapped[тип]** задает тип данных для аннотаций, а **mapped_column()** конфигурирует ограничения на уровне СУБД.
4. в) Очень удобный метод «всё или ничего». Он не трогает уже созданные таблицы, но генерирует недостающие по текущему коду моделей.
5. б) Использование **with** избавляет разработчика от необходимости писать **try...finally** блоки для закрытия соединений при ошибках.

6. `from sqlalchemy import create_engine`

7. Код:

```
from sqlalchemy.orm import DeclarativeBase
```

```
class Base(DeclarativeBase):
```

```
    pass
```

```
8. email: Mapped[str] = mapped_column(unique=True,  
    nullable=False)
```

```
9. session.commit()
```

```
10. sqlite:///app.db (три слэша означают относительный путь к  
    файлу в текущей директории).
```


Тест 3: CRUD-операции и связи в SQLAlchemy

1. в) В версии 2.0 подход изменился: сначала строится объект запроса через `select()`, а затем передается на выполнение в сессию.
2. б) `session.get(Model, id)` – самый короткий путь. Он сначала заглядывает во внутренний кэш сессии, избегая лишнего запроса к диску, если объект уже загружен.
3. б) `relationship()` – чисто ORM-инструмент для удобной навигации по объектному графу в коде Python.
4. в) Изменения пропадут. Сессия сбросит незафиксированные данные, сделав автоматический `rollback`.
5. б) По умолчанию `session.execute()` возвращает строки-кортежи. Метод `.scalars()` превращает их в плоский список объектов модели.
6. `select(User).where(User.role == 'admin')`
7. `.order_by(Product.price.desc())` (не забудьте импортировать функцию `desc` из пакета `sqlalchemy`).
8. `session.delete(order)`
9. `back_populates` (указывается строковое имя атрибута на другой модели).
10. `select(Model).offset(10).limit(5)` (сначала отсекаем пропущенное, затем берем лимит).